

WOJSKOWA AKADEMIA TECHNICZNA

im. Jarosława Dąbrowskiego

WYDZIAŁ CYBERNETYKI

Sprawozdanie

z zajęć laboratoryjnych z przedmiotu

Teoria informacji i kodowania

Temat: **Implementacja**

kodera/dekodera słownikowego

metodą LZ77 i implementacja funkcji

CRC

Sprawozdanie wykonał: Godlevskyi Andrii

Numer grupy: WCY23KY1S1

Prowadzący: dr inż. Leszek Grad

Treść zadania:

Implementacja kodera/dekodera słownikowego. Metoda: LZ77

Etapy:

1. Funkcja: Wyszukiwanie najdłuższej sekwencji z bufora w słowniku.
2. Implementacja kodera (wykorzystanie funkcji z p.1, operacje plikowe)
3. Implementacja dekodera.
4. Implementacja funkcji CRC32

Analiza zadania:

1. Funkcja wyszukiwania najdłuższej sekwencji z bufora w słowniku.
 - Cel: Porównanie i znalezienie danych w buforze dla zwrócenia najdłuższego ciągu, który pasuje do sekwencji w słowniku.
 - Realizacja: Iteracyjne porównywanie zawartości bufora ze słownikiem, identyfikacja odległości, długości sekwencji oraz następnego symbolu.
2. Implementacja kodera
 - Cel: Kompresja danych za pomocą funkcji z 1 punktu.
 - Realizacja: Wczytywanie danych, użycie algorytmu LZ77 dla kompresowania danych trójkami (odległość, długość, symbol), zapis skompresowanych danych do pliku.
3. Implementacja dekodera:
 - Cel: Dekompresja zakodowanych przez koder danych.
 - Realizacja: Wczytywanie skompresowanych danych, odtwarzanie oryginalnych danych na podstawie trójek (odległość, długość, symbol), zapis zdekompresowanych danych do pliku.
4. Implementacja funkcji CRC32
 - Cel: Dodanie funkcji CRC32 do programu implementującego algorytm LZ77 w celu weryfikacji integralności danych.
 - Realizacja:
 1. W koderze: Obliczanie sumy kontrolnej CRC32 na podstawie skompresowanych danych, dopisywanie wartości CRC32 do pliku wyjściowego po kompresji danych wejściowych.
 2. W dekodерze: Wczytanie danych oraz sumy CRC32 przed procesem dekodowania, ponowne obliczenie CRC32 na podstawie wczytanych danych i porównanie z zapisaną w pliku wejściowym wartością w celu weryfikacji integralności, jeśli wartości się różnią, dane są uszkodzone.

Założenia:

- Program realizuje kompresję i dekompresję danych metodą LZ77, która jest metodą bezstratnej kompresji, dane wejściowe (w pliku tekstowym lub binarnym) będą kompresowane przez koder do pliku binarnego, natomiast dekodер przywróci je do pierwotnej postaci, projekt będzie zoptymalizowany pod kątem szybkości działania oraz efektywności kompresji i dekompresji.

- Program wykorzystuje dwa bufory: Bufor słownika i bufor wyprzedzenia. Rozmiary obu buforów są określone przez użytkownika podczas działania programu i od ich wartości zależy efektywność kompresji danych.
- Implementacja funkcji CRC32 w koder i dekodery zapewnia spójność danych między procesami kodowania i dekodowania, pliki wejściowe i wyjściowe muszą być prawidłowo obsługiwane, z uwzględnieniem potencjalnych błędów, takich jak brak dostępu do pliku czy uszkodzenie danych.

Algorytm kodera:

1. Przygotowanie struktury Match i inicjalizacja zmiennych
 - Definicja struktury zawierającej informacje o jednym dopasowaniu (offset,length,nextChar)
 - Inicjalizacja takich zmiennych jak długość bufora, słownika i t.d.
2. Wczytanie parametrów i danych wejściowych
 - Pobieranie danych od użytkownika stosownie plików wejściowych i wartości bufora wraz ze słownikiem
 - Wczytanie zawartości pliku wejściowego z danymi do skompresowania
3. Procedura kompresji
 - 1) Wypełnianie bufora danymi do kompresji
 - 2) Poszukiwanie najdłuższego dopasowania ciągu danych w buforze porównując z danymi w słowniku (funkcja findLongestMatch)
 - 3) Dodanie wyniku do wektora dopasowań (compressed)
 - 4) Aktualizacja bufora i słownika
 - 5) Powtarzanie procesów 2-4 aż do końca pliku z danymi wejściowymi
4. Zbieranie i kodowanie wyników
 - 1) Tworzenie tymczasowego bufora w pamięci (memBuf) do przechowywania wyników
 - 2) Kodowanie danych
 - 3) Uzupełnienie bufora kolejnymi danymi do kodowania
 - 4) Powtarzanie procesów 2-3 do wyczerpania danych
 - 5) Obliczanie sumy kontrolnej CRC dla całego bufora (funkcja computeCRC32)
5. Zapis danych do pliku i dodanie CRC
 - 1) Otwieranie pliku wyjściowego
 - 2) Zapisanie do niego całej zawartości bufora tymczasowego (memBuf)
 - 3) Dopisanie 4 bajtów obliczonej sumy CRC32
 - 4) Zamknięcie pliku wyjściowego
6. Obliczanie stopnia kompresji
 - 1) Odczytanie rozmiarów pliku wejściowego a wyjściowego
 - 2) Wyznaczenie współczynnika kompresji
7. Prezentacja wyników
 - Obejrzenie wektora compressed i wypisanie w konsoli wszystkich wartości
 - Wypisanie stopnia kompresji

8. Zakończenie
 - Zamknięcie programu i zwalnianie zasobów

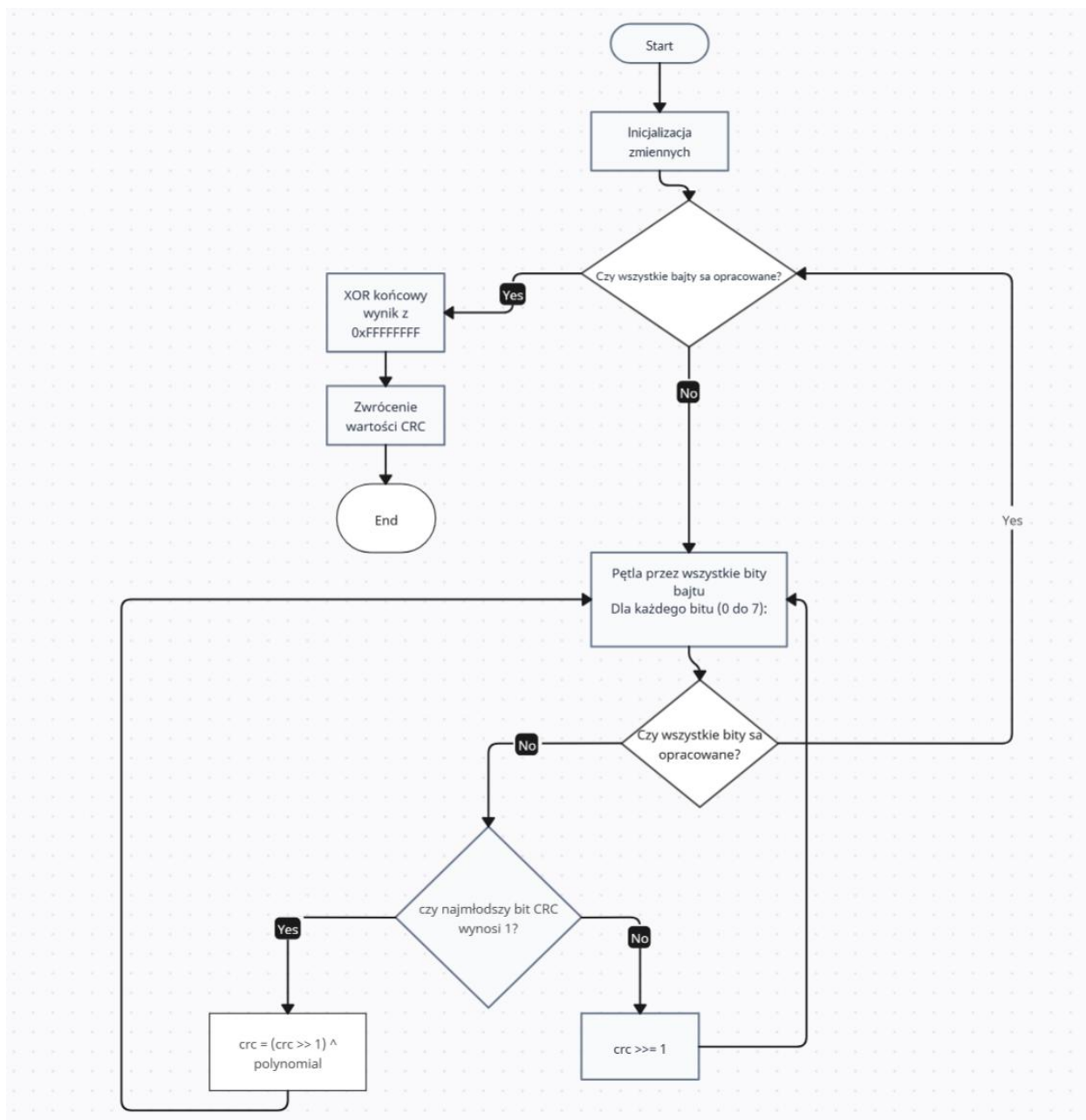
Algorytm dekodera:

1. Przygotowanie struktury Match i inicjalizacja zmiennych
 - Definicja struktury zawierającej informacje o jednym dopasowaniu (offset,length,nextChar)
 - Inicjalizacja takich zmiennych jak długość bufora, słownika i t.d.
2. Wczytanie parametrów i danych wejściowych
 - Pobieranie danych od użytkownika stosownie plików wejściowych i wartości bufora wraz ze słownikiem (takie same jak przy kodowaniu)
 - Wczytanie zawartości pliku wejściowego z danymi do dekompresowania
3. Sprawdzenie CRC
 - 1) Oddzielenie ostatnich 4 bajtów danych (CRC) w pliku i zapamiętanie je w zmiennej (readCRC),część z wyłączonej danymi skompresowanymi jest zapisywana oddzielnie (compData)
 - 2) Ponowne obliczenie CRC dla compData (funkcja computeCRC32)
 - 3) Porównanie obliczonego ponownie CRC z wartością z pliku
4. Dekompresja
 - 1) Odczytanie wczytanych z pliku wartości (już bez CRC)
 - 2) Zapisanie odczytanych trójek w wektorze
 - 3) Powtarzanie procesów 1-2 do wyczerpania danych
 - 4) Dekodowanie danych dla wszystkich elementów w wektorze
 - 5) Kopiowanie znaków na koniec wyjściowego łańcucha (decompressed)
5. Zapis zdekodowanych danych do pliku
 - 1) Otwieranie pliku wyjściowego
 - 2) Zapisanie do niego całego łańcucha zdekodowanych danych (decompressedData)
 - 3) Zamknięcie pliku wyjściowego
6. Prezentacja wyników
 - Wyświetlenie komunikatu o poprawnym wykonaniu dekompresji.
 - Wypisanie zdekodowanych danych
7. Zakończenie
 - Zamknięcie programu i zwalnianie zasobów

Implementacja:

Cały kod został zaimplementowany w języku C++ i stanowi dobry przykład podstawowej kompresji danych za pomocą algorytmu LZ77 oraz dodatkowo pokazuję jeden ze sposobów sprawdzenia integralności danych który jest stosowany w prawdziwej branży. Dla testów dopisałem funkcje w programie kodującym, która oblicza stopień kompresji danych i działa poprzez porównanie rozmiaru oryginalnego pliku z rozmiarem pliku po kompresji. Kod dekodera różni się od postaci wysłanej na Teams bo przerobiłem go na sposób bardziej optymalny ta efektywny.

Algorytm CRC w postaci diagramu:



To nie jest idealny diagram algorytmu crc ale mniej więcej przedstawia jego postać.

Testowanie poprawności działania programu:

1. Plik zawierający ciąg losowych cyfr:

Zawartość pliku wejściowego: 909175640314171177007132

Wynik działania kodera:

```
Podaj nazwe pliku wejsciowego: we.txt
Podaj nazwe pliku wyjsciowego: wy.bin
Podaj rozmiar słownika: 14
Podaj rozmiar bufora: 7
Stopień kompresji: 1.91667
(0, 0, 9)
(0, 0, 0)
(2, 1, 1)
(0, 0, 7)
(0, 0, 5)
(0, 0, 6)
(0, 0, 4)
(7, 1, 3)
(7, 1, 4)
(9, 2, 1)
(12, 2, 7)
(10, 1, 0)
(7, 2, 3)
(0, 0, 2)
```

Wynik działania dekodera:

```
Podaj nazwe pliku wejsciowego: wy.bin
Podaj nazwe pliku wyjsciowego: wynik.txt
Podaj rozmiar słownika (ten sam co podczas kompresji): 14
Podaj rozmiar bufora (ten sam co podczas kompresji): 7
Dekompresja zakończona poprawnie.
Odtworzony tekst:
909175640314171177007132
```

Zawartość pliku wyjściowego: 909175640314171177007132

Wynik w pełni się zgadza z początkowymi danymi, więc dla tego przykładu program działa poprawnie.

2. Plik zawierający ciąg losowych małych liter:

Zawartość pliku wejściowego: badcdefabacdacabcbaddcbaef

Wynik działania kodera:

```
Podaj nazwe pliku wejsciowego: we.txt
Podaj nazwe pliku wyjsciowego: wy.bin
Podaj rozmiar słownika: 24
Podaj rozmiar bufora: 12
Stopień kompresji: 1.72
(0, 0, b)
(0, 0, a)
(0, 0, d)
(0, 0, c)
(2, 1, e)
(0, 0, f)
(6, 1, b)
(6, 2, a)
(9, 1, a)
(6, 2, b)
(16, 2, d)
(5, 3, e)
(18, 1, )
```

Wynik działania dekodera:

```
Podaj nazwe pliku wejsciowego: wy.bin
Podaj nazwe pliku wyjsciowego: wyn.txt
Podaj rozmiar słownika (ten sam co podczas kompresji): 24
Podaj rozmiar bufora (ten sam co podczas kompresji): 12
Dekompresja zakończona poprawnie.
Odtworzony tekst:
badcdefabacdacabcbaddcbaef
```

Zawartość pliku wyjściowego: badcdefabacdacabcbaddcbaef

Wynik w pełni się zgadza z początkowymi danymi ,więc dla tego przykładu program działa poprawnie.

3. Plik zawierający losowy ciąg znaków:

Zawartość pliku wejściowego: pGkiMnJCtU2ZFLhgr3odM9jtB0Xqfy8evdGD6R5u7Fck9DsEs

Wynik działania Kodera:

```
Podaj nazwe pliku wejsciowego: we.txt
Podaj nazwe pliku wyjsciowego: wy.bin
Podaj rozmiar słownika: 50
Podaj rozmiar bufora: 45
Stopień kompresji: 2.66
(0, 0, p)
(0, 0, G)
(0, 0, k)
(0, 0, i)
(0, 0, M)
(0, 0, n)
(0, 0, J)
(0, 0, C)
(0, 0, t)
(0, 0, U)
(0, 0, 2)
(0, 0, Z)
(0, 0, F)
(0, 0, L)
(0, 0, h)
(0, 0, g)
(0, 0, r)
(1, 1, 3)
(0, 0, o)
(0, 0, d)
(17, 1, 9)
(0, 0, j)
(16, 1, B)
(0, 0, 0)
(0, 0, X)
(0, 0, q)
(0, 0, f)
(0, 0, y)
(0, 0, 8)
(0, 0, e)
(0, 0, v)
(14, 1, G)
(0, 0, D)
(0, 0, 6)
(0, 0, R)
(0, 0, 5)
(0, 0, u)
(0, 0, 7)
(30, 1, c)
(42, 1, 9)
(10, 1, s)
(0, 0, E)
(2, 1, )
```


Wynik działania dekodera:

```
Podaj nazwe pliku wejscowego: wy.bin
Podaj nazwe pliku wyjscowego: wynik.txt
Podaj rozmiar slownika (ten sam co podczas kompresji): 50
Podaj rozmiar bufora (ten sam co podczas kompresji): 45
Dekompresja zakonczona poprawnie.
Odtworzony tekst:
pGkiMnJCtU2ZFLhgrrr3odM9jtB0Xqfy8evdGD6R5u7Fck9DsEs
```

Zawartość pliku wyjściowego: pGkiMnJCtU2ZFLhgrrr3odM9jtB0Xqfy8evdGD6R5u7Fck9DsEs

Wynik w pełni się zgadza z początkowymi danymi ,więc dla tego przykładu program działa poprawnie.

Dla tego samego pliku sprawdzam poprawność działania CRC, w pliku wyjściowym wy.bin zmieniam jeden dowolny znak żeby sprawdzić wykrycie błędu, wynik działania dekodera w takim razie:

```
Podaj nazwe pliku wejscowego: wy.bin
Podaj nazwe pliku wyjscowego: wynik.txt
Podaj rozmiar slownika (ten sam co podczas kompresji): 50
Podaj rozmiar bufora (ten sam co podczas kompresji): 45
BLAD: plik uszkodzony! CRC sie nie zgadza.
```

Wnioskując ,program działa poprawnie i wykrywa gdy CRC z pliku i ponownie obliczone się nie zgadzają.

Testowanie stopnia kompresji dla różnego typu danych:

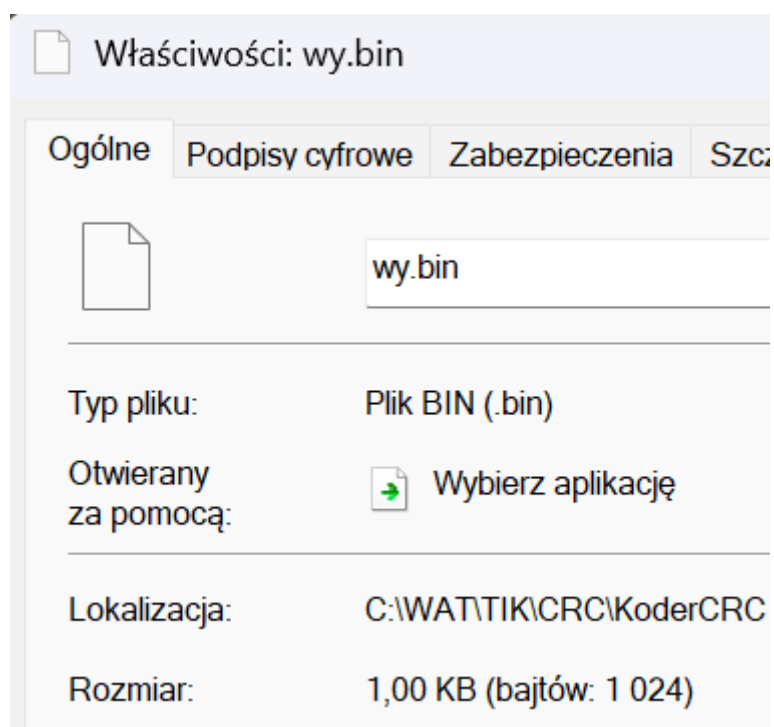
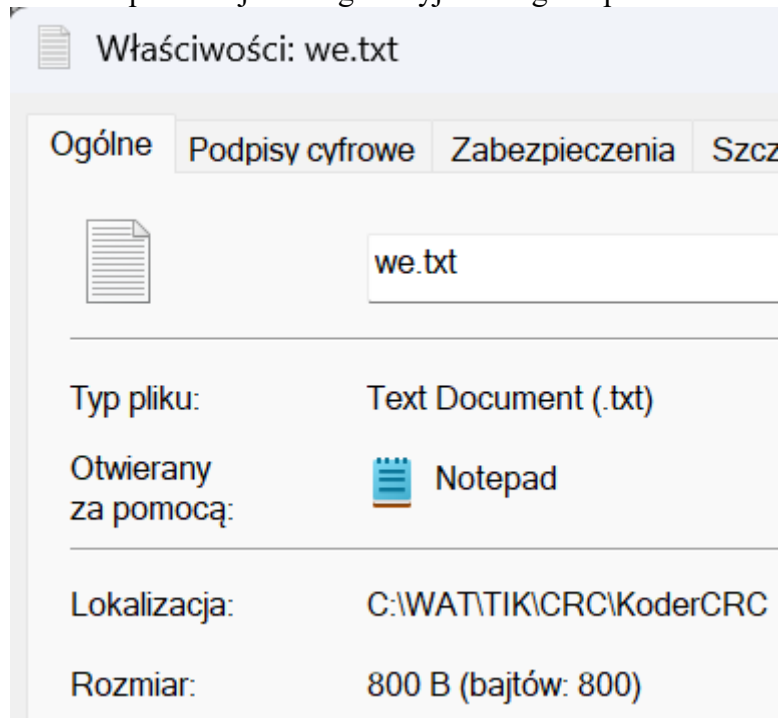
1. Kompresowanie pliku zawierającego 800 losowo umieszczonych liter a,m,l,e,g,f

Zawartość pliku:

Mmalnfemfafaemgafeaggelaaaemlleaaamgegfflgggllaefflmfmgflfgegfmegflgfemmlmaamlgel
lgammmemmagmmmfefgglggmegmgmmammgaggeeafllglleafflglamgaleflalmfemfemlmemf
ggmmflffgefafmgegeeeeglgafafmagmeeefafmfemgeggmlfafggfmfmglfgmaflamagfflffmgmgflaef
amfaffafeelglfelgmlaffllmgfammlaeleaaglgemmlglflgaamlfalggfefflaefemaaalaggfemfelefflg
agegamgegffleamgmfgaelaamaeagalgalmlaeeglgalgmggallealmflamamememgflglgelemmaaem
mlellmeagmmmggflmagalmeglfefelafmfemaellammllmfemefggfalgmfamffamfmeghmaamlmea
fflllgemlaamgagglfmfeafemmfggfellaafggfmfaelamgfefmelgaealfaeaglfmmeammagglfalmgff
gmegflfallfgmemaamggaegmfafgaaglfeafaelleaeafamgeffmgefllgffgmefalgaglemfamflgfffgfm
mgmffmfmglllfeemgfefeefafaaafalfamggflaeelfeeegfmemmmfelffamaegeeegmmflgfaamlgm
maaeaeamllffgamllmaagfmelmglemellglelmmmgammlfamg

Wartość słownika = 512 ,bufora = 256

Rozmiar pliku wejściowego i wyjściowego odpowiednio:



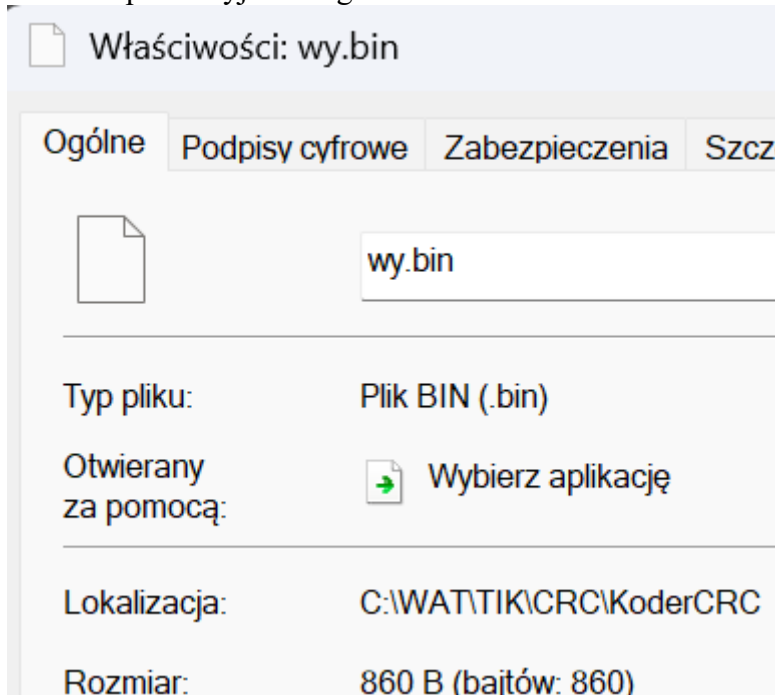
Obliczony stopień kompresji:

```
Podaj nazwe pliku wejsciowego: we.txt
Podaj nazwe pliku wyjsciowego: wy.bin
Podaj rozmiar słownika: 512
Podaj rozmiar bufora: 256
Stopien kompresji: 1.28
```

Rozmiar pliku wyjściowego jest większy od wejściowego ,więc spróbuję dobrać inne wartości bufora uwzględniając częstość występowania ciągów liter.

Podaję wartość słownika = 256 ,bufora = 128

Rozmiar pliku wyjściowego:



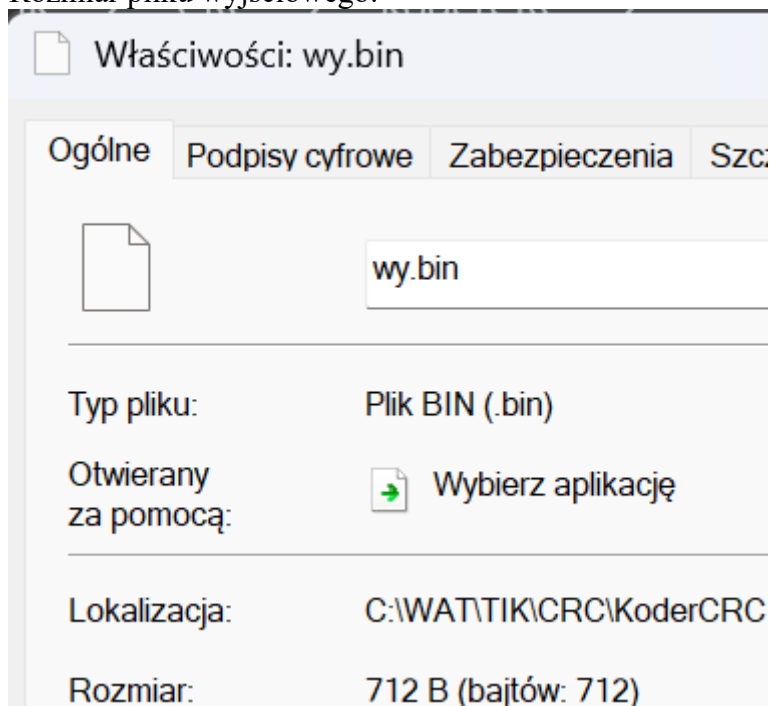
Stopień kompresji:

```
Podaj nazwe pliku wejsciowego: we.txt
Podaj nazwe pliku wyjsciowego: wy.bin
Podaj rozmiar słownika: 256
Podaj rozmiar bufora: 128
Stopien kompresji: 1.075
```

Dostaliśmy o wiele lepszy wynik ale dalej plik wyjściowy jest większy, próbuję dalej zmieniać rozmiar bufora i słownika.

Podaję wartość słownika = 128 ,bufora = 56

Rozmiar pliku wyjściowego:



Stopień kompresji:

```
Podaj nazwe pliku wejscowego: we.txt
Podaj nazwe pliku wyjscowego: wy.bin
Podaj rozmiar słownika: 128
Podaj rozmiar bufora: 56
Stopień kompresji: 0.89
```

Wniosek:

Końcowy wynik można uznać za efektywny, plik po kompresji ma mniejszy rozmiar od początkowego, dobierać jeszcze mniejsze wartości słownika i bufora ale to nie udało mi się dostać jeszcze bardziej lepszego stopnia kompresji. Związane to jest z tym że dane nie powtarzają się aż tak często, a efektywność LZ77 w pełni zależy od długości powtarzających się ciągów.

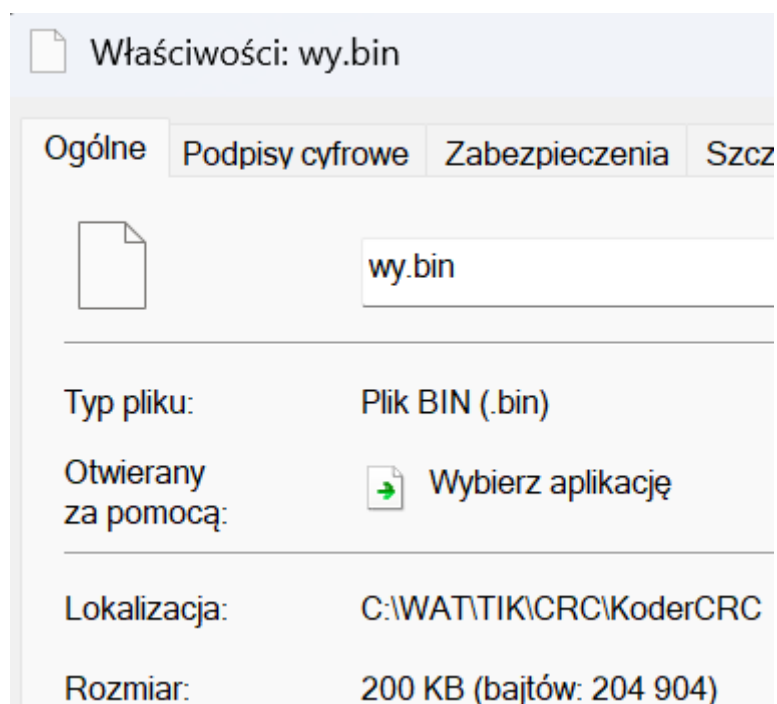
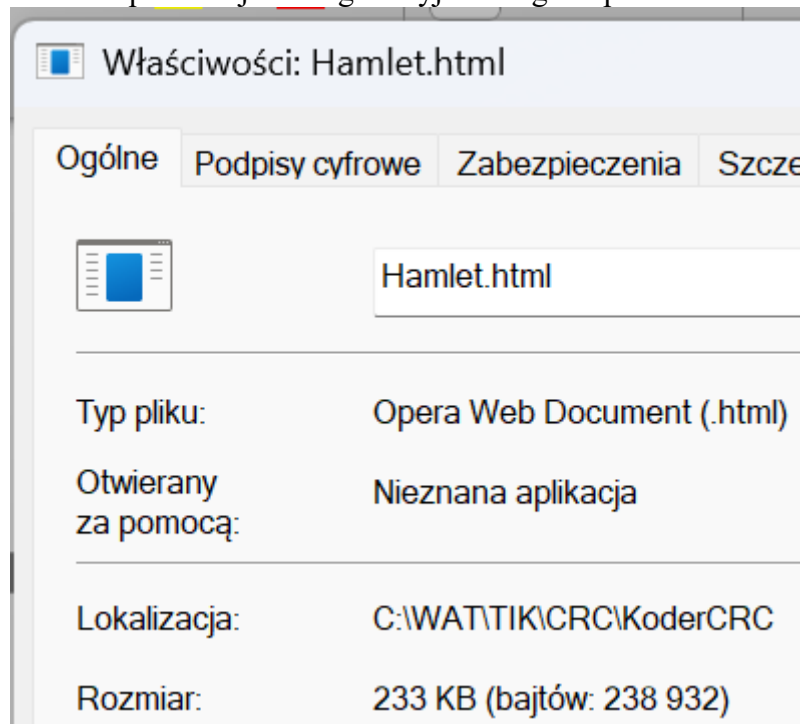
2. Kompresowanie całego tekstu Hamlet William Shakespeare

Wyłączyłem tu funkcje wypisania danych w postaci trójek
(// printCompressedData(compressed);) dla lepszej widoczności

W tym teście sprawdzam czy działa kompresja po pierwsze na plikach nie .txt a .html (może być jakikolwiek inny plik) a po drugie na plikach o dużych rozmiarach.

Wartość słownika = 4096 ,bufora = 2048

Rozmiar pliku wejściowego i wyjściowego odpowiednio:

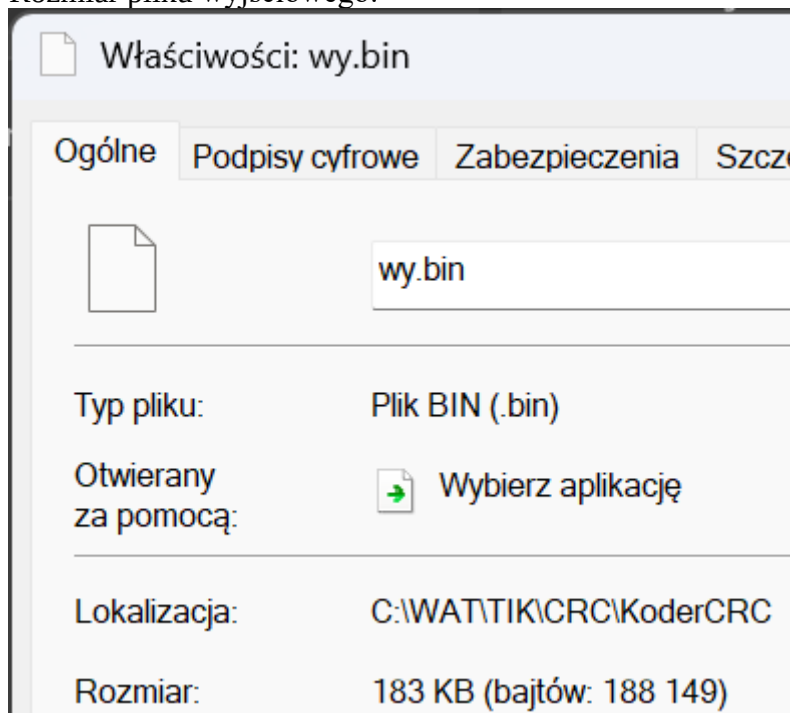


Stopień kompresji:

```
Podaj nazwe pliku wejsciowego: hamlet.html
Podaj nazwe pliku wyjsciowego: wy.bin
Podaj rozmiar słownika: 4096
Podaj rozmiar bufora: 2048
Stopien kompresji: 0.857583
```

Zmieniając wartości słownika i bufora odpowiednio na 8192 i 4096 uzyskuję taki wynik:

Rozmiar pliku wyjściowego:



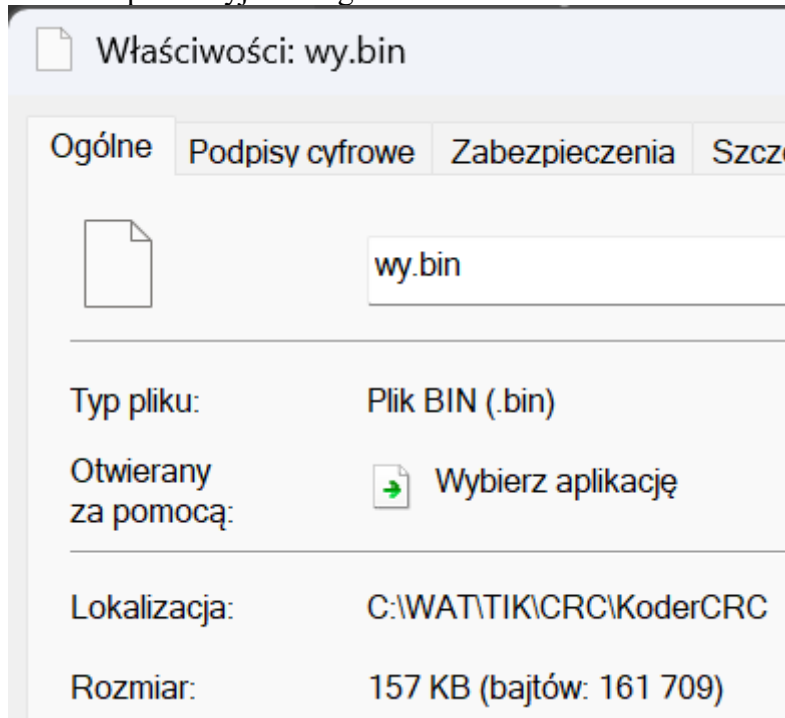
Stopień kompresji:

```
Podaj nazwe pliku wejsciowego: Hamlet.html
Podaj nazwe pliku wyjsciowego: wy.bin
Podaj rozmiar słownika: 8192
Podaj rozmiar bufora: 4096
Stopień kompresji: 0.787458
```

Próbuję użyć jeszcze większe liczby dla słownika i bufora ,32768 i 16384 odpowiednio

Wynik:

Rozmiar pliku wyjściowego:



Stopień kompresji:

```
Podaj nazwe pliku wejscowego: Hamlet.html
Podaj nazwe pliku wyjscowego: wy.bin
Podaj rozmiar słownika: 32768
Podaj rozmiar bufora: 16384
Stopień kompresji: 0.676799
```

Wniosek:

Przy kodowaniu długich plików tekstowych takich jak książki zwiększenie bufora i słownika jest kluczowe dla efektywnej kompresji dlatego, że ponieważ pozwala na obsługiwanie większych bloków tekstów i podwyższając szanse na znalezienie dłuższej sekwencji.

1. Kompresowanie ciągu 1600 losowych pseudoloswych znaków.

Zawartość pliku wejściowego:

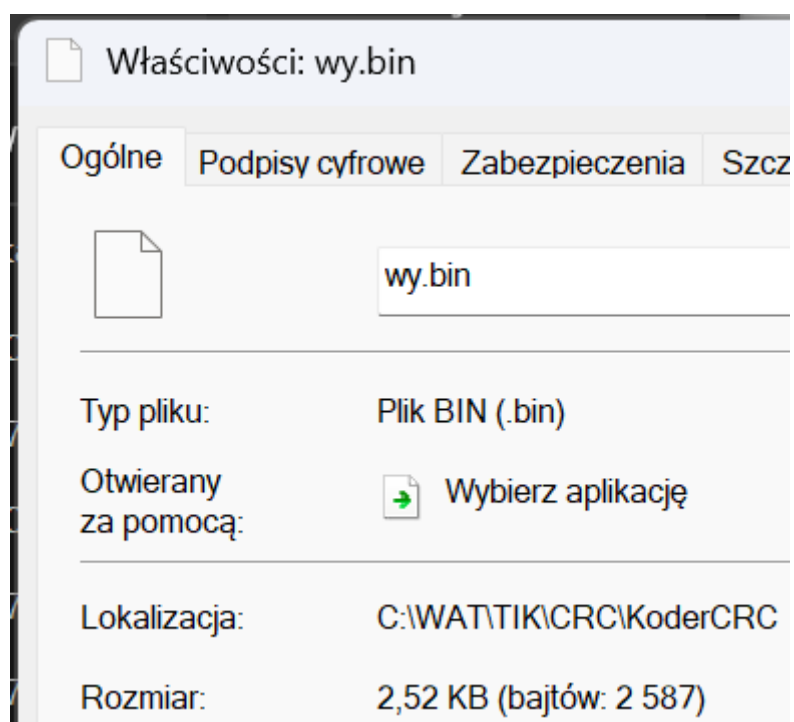
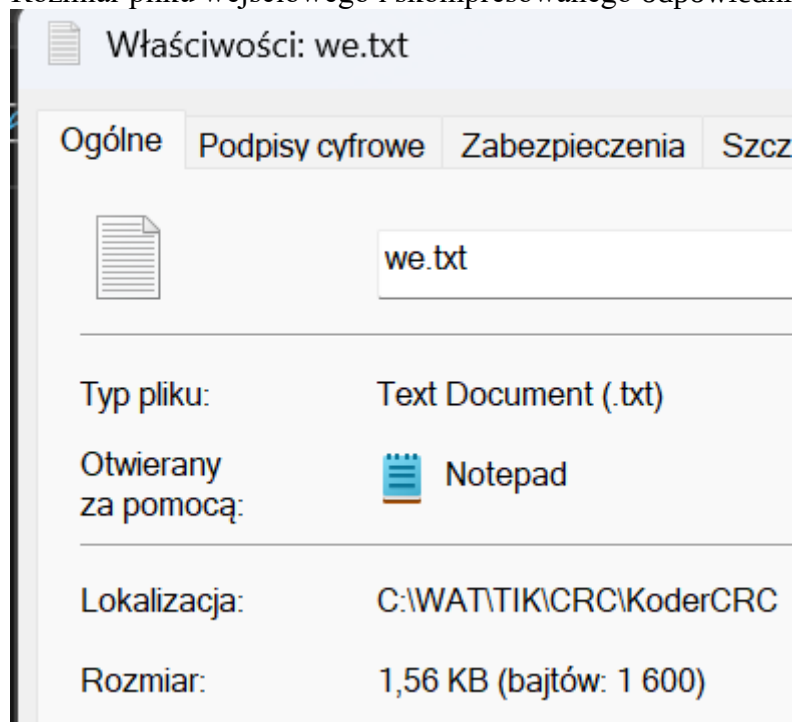
y8@aJOFg1TXYPs@elv4Tu3UHNer!4=dYRSrbdIL0%0UMZD6Tm%K=Qp4+jn&3Cd\$\$tivL
VM0Izd\$=rXIWomBiphkx31_H&hAKiw#avEtUgR+A9*K=G=tf87w5%m@*9U4cQORM^5fn
do0&Z^O2yj^8NY3VWR7B32y8jB_t7s#u=@AQ#JkQTF&aGzd7=P\$UtSqBJiNwawOz@sjV3
qT=+*D5=QQE7Htdy_cuTFaolr-
0U+NWKSwe1m7t4IgC!X35hy8@aJOFg1TXYPs@elv4Tu3UHNer!4=dYRSrbdIL0%0UMZ
D6Tm%K=Qp4+jn&3Cd\$\$tivLVM0Izd\$=rXIWomBiphkx31_H&hAKiw#avEtUgR+A9*K=G=
tf87w5%m@*9U4cQORM^5fndo0&Z^O2yj^8NY3VWR7B32y8jB_t7s#u=@AQ#JkQTF&aGz
d7=P\$UtSqBJiNwawOz@sjV3qT=+*D5=QQE7Htdy_cBJiNwawOz@sjV3qT=+*D5=QQE7Ht
dy_cuTFaolr-0U+NWKSwe1m7t4IgC!X35hFXWy_bbJa6kO&D*J3brs^NXILmV%dizLP-
E@R2=*N1+f*^En^EnROBEeFa=x41utRVCC&sfwhupB#YT!kAf1G&i_j%iP4AjZ0s1sfTq^L#
M=yXEY1rmOH%N&E-
ddx9Zclra&!onaD*r3mZ#f3RT+NWKSwe1m7t4IgC!X35hFXWy_bbJa6kO&D*J3brs^NXILm
V%dizLP-
E@R2=*N1+f*^En^EnROBEeFa=x41utRVCC&sfwhupB#YT!kAf1G&i_j%iP4AjZ0s1sfTq^L#
M=yXEY1rmOH%N&E-
ddx9Zclra&!onaD*r3mZ#f3RTFXWy_bbJa6kO&D*J3brs^NXILmV%dizLP-
E@R2=*N1+f*^En^EnROBEeFa=x41utRVCC&sfwhupB#YT!kAf1G&i_j%iP4AjZ0s1sfTq^L#
M=yXEY1rmOH%N&E-ddx9Zclra&!onaD*r3mZ#f3RTuTFaolr-
0Uy8@aJOFg1TXYPs@elv4Tu3UHNer!4=dYRSrbdIL0%0UMZD6Tm%K=Qp4+jn&3Cd\$\$ti
vLVM0Izd\$=rXIWomBiphkx31_H&hAKiw#avEtUgR+A9*K=G=tf87w5%m@*9U4cQORM^
5fndo0&Z^O2yj^8NY3VWR7B32y8jB_t7s#u=@AQ#JkQTF&aGzd7=P\$UtSqBJiNwawOz@sj
V3qT=+*D5=QQE7Htdy_cuTFaolr-
0U+NWKSwe1m7t4IgC!X35hFXWy_bbJa6kO&D*J3brs^NXILmV%dizLP-
E@R2=*N1+f*^En^EnROBEeFa=x41utRVCC&sfwhupB#YT!kAf1G&i_j%iP4AjZ0s1sfTq^L#
M=yXEY1rmOH%N&E-
ddx9Zclra&!onaD*r3mZ#f3RT+NWKSwe1m7t4IgC!X35hFXWy_bbJa6kO&D*J3brs^NXILm
V%dizLP-
E@R2=*N1+f*^En^EnROBEeFa=x41utRVCC&sfwhupB#YT!kAf1G&i_j%iP4AjZ0s1sfTq^L#
M=yXEY1rmOH%N&E-ddx9Zclra&!onaD*r3mZ#f3RTFXWy_bbJa6kO&D*J3brs^NXI

Początkowe wartości :słownika=128, buforu=56

Stopień kompresji:

```
Podaj nazwe pliku wejsciowego: we.txt
Podaj nazwe pliku wyjsciowego: wy.bin
Podaj rozmiar słownika: 128
Podaj rozmiar bufora: 56
Stopien kompresji: 1.61688
```

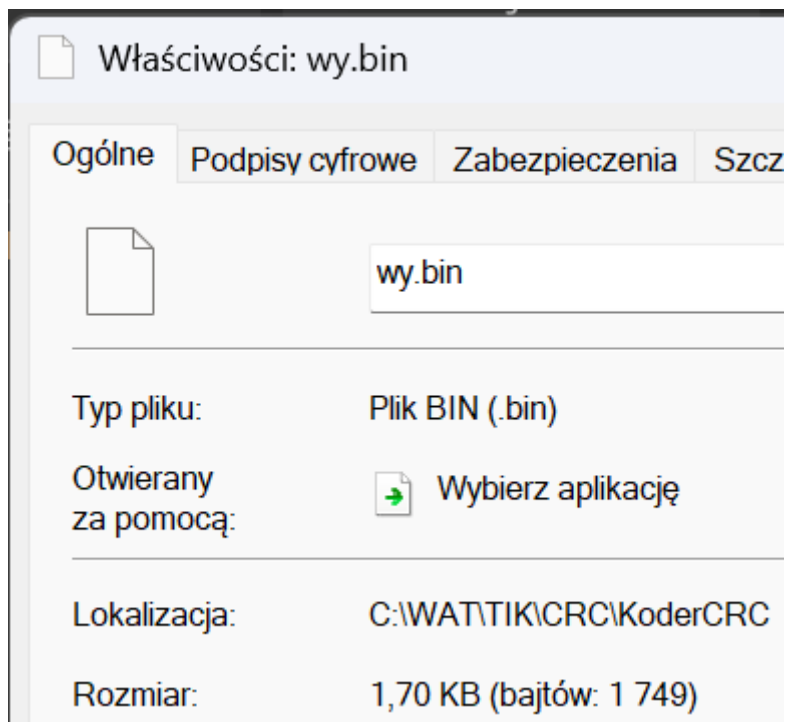

Rozmiar pliku wejściowego i skompresowanego odpowiednio:



Zwiększam słownik do 512 a bufor do 256

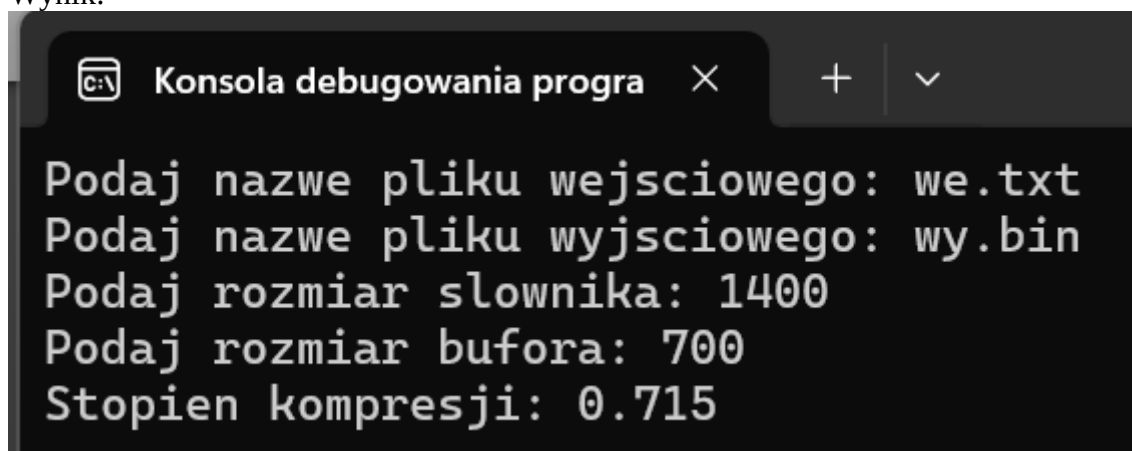
Wynik:

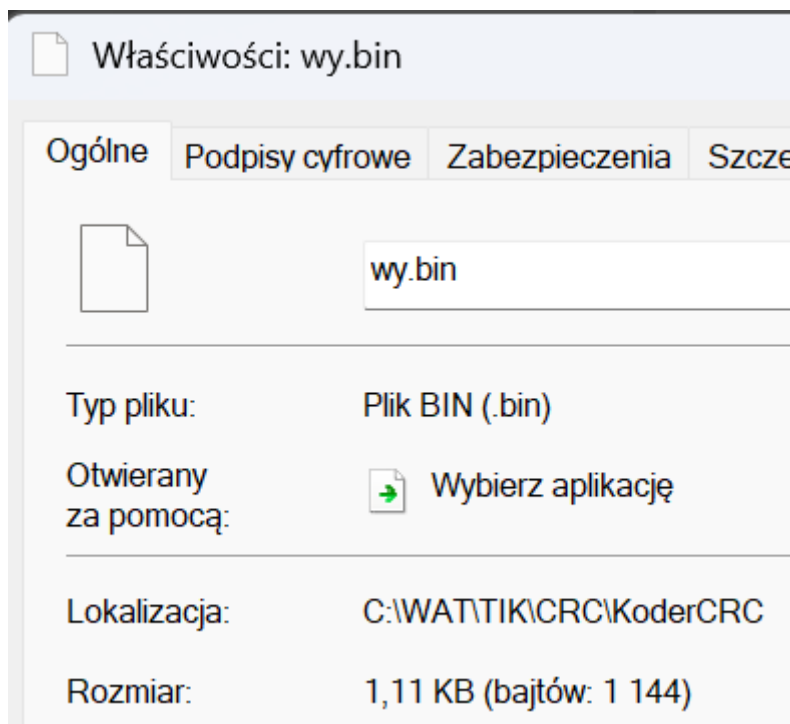
```
Podaj nazwe pliku wejscowego: we.txt
Podaj nazwe pliku wyjscowego: wy.bin
Podaj rozmiar słownika: 512
Podaj rozmiar bufora: 256
Stopien kompresji: 1.09312
```



Próbuję użyć jeszcze większych rozmiarów, 1400 i 700 odpowiednio

Wynik:





Wniosek:

Oczekiwałem że kompresja będzie nieefektywna, ale z powodu tego że ciąg jest pseudolosowy LZ77 wykrył powtarzające się ciągi przy dużym rozmiarze słownika i bufora, czyli dostaliśmy podobny do kodowania książek efekt.

Podsumowanie:

Algorytm LZ77 jest najbardziej efektywny w przypadkach, gdy dane zawierają powtarzające się sekwencje, co jest typowe dla tekstów. W przypadku danych o wysokiej entropii, na przykład losowe znaki, algorytm ten działa znacznie gorzej, co często skutkuje zwiększeniem rozmiaru po kompresji.

Wnioskując najbardziej znaczące czynniki to rozmiar słownika, bufora i wejściowe dane, od rozmiarów zależy możliwość wyszukania najlepszych dopasowań, a od wejściowych danych czy jest w ogóle możliwość znaleźć dopasowanie.

Kod źródłowy programów:

Koder:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
#include <cmath>
#include <bitset>
#include <string>
#include <stdint>

using namespace std;

// Struktura dopasowania
struct Match {
    int offset;    // Przesunięcie w słowniku
    int length;    // Długość dopasowania
    char nextChar; // Następny znak
};

// Obliczenie CRC32
uint32_t computeCRC32(const unsigned char* data, size_t length) {
    uint32_t crc = 0xFFFFFFFF;
    uint32_t polynomial = 0xEDB88320; // typowy wielomian

    for (size_t i = 0; i < length; ++i) {
        crc ^= data[i];
        for (int b = 0; b < 8; b++) {
            if (crc & 1) {
                crc = (crc >> 1) ^ polynomial;
            }
            else {
                crc >>= 1;
            }
        }
    }
    return crc ^ 0xFFFFFFFF;
}

// Funkcja kodująca wartość z użyciem zmiennej liczby bitów
void encodeVariableLength(ofstream& outputFile, int value, int maxValue) {
    int numBits = (maxValue > 0) ? (int)ceil(log2(maxValue + 1)) : 1;
    int byteCount = (numBits + 7) / 8;
    for (int i = 0; i < byteCount; i++) {
        char c = (char)((value >> (8 * i)) & 0xFF);
        outputFile.write(&c, 1);
    }
}

// Wyszukiwanie najdłuższej sekwencji z bufora w słowniku
```

```

Match findLongestMatch(const char* dictionary, const char* buffer,
    int dictLength, int bufferLength) {
    Match match{};
    match.offset = 0;
    match.length = 0;
    match.nextChar = 0;

    for (int i = 0; i < dictLength; i++) {
        int j = 0;
        while (j < bufferLength &&
            dictionary[i + j] == buffer[j] &&
            (i + j) < dictLength) {
            j++;
        }
        if (j > match.length) {
            match.offset = dictLength - i;
            match.length = j; // Jesli j < bufferLength, to jest kolejny znak
            // Jesli j == bufferLength, to znaczy, ze caly bufor jest dopasowany, brak dodatkowego
            // znaku
            if (j < bufferLength) {
                match.nextChar = buffer[j];
            }
            else {
                // Pelne dopasowanie bufora, brak nastepnego znaku
                match.nextChar = 0;
            }
        }
    }
    return match;
}

```

// Funkcja kompresji algorytmem LZ77

```

vector<Match> compressLZ77(const string& text, int dictSize, int bufferSize) {
    vector<Match> compressed;
    int i = 0;
    while (i < (int)text.size()) {
        int dictStart = max(0, i - dictSize);
        int dictLength = i - dictStart;
        int bufferLength = min(bufferSize, (int)text.size() - i);

        Match match = findLongestMatch(
            text.c_str() + dictStart,
            text.c_str() + i,
            dictLength,
            bufferLength);

        if (match.length == 0) {
            match.nextChar = text[i];
        }
        compressed.push_back(match);
        i += (match.length > 0) ? (match.length + 1) : 1;
    }
}

```

```

    return compressed;
}

// Funkcja zapisu, drukowania i dynamicznego obliczenia CRC
void saveAndPrintCompressed(const vector<Match>& compressed,
    const string& outputFileName,
    int dictSize, int bufferSize) {
    ofstream outputFile(outputFileName, ios::binary);
    if (!outputFile.is_open()) {
        cerr << "Nie mozna otworzyc pliku wyjsciowego: "
            << outputFileName << endl;
        return;
    }

    // Zbieramy dane do wektora w pamieci, by na koncu policzyc CRC
    vector<unsigned char> memBuf;
    memBuf.reserve(compressed.size() * 10);

    for (const Match& match : compressed) {
        // offset
        {
            int value = match.offset;
            int numBits = (dictSize > 0)
                ? (int)ceil(log2(dictSize + 1)) : 1;
            int byteCount = (numBits + 7) / 8;
            for (int i = 0; i < byteCount; i++) {
                memBuf.push_back((unsigned char)((value >> (8 * i)) & 0xFF));
            }
        }
        // length
        {
            int value = match.length;
            int numBits = (bufferSize > 0)
                ? (int)ceil(log2(bufferSize + 1)) : 1;
            int byteCount = (numBits + 7) / 8;
            for (int i = 0; i < byteCount; i++) {
                memBuf.push_back((unsigned char)((value >> (8 * i)) & 0xFF));
            }
        }
        // nextChar
        memBuf.push_back((unsigned char)match.nextChar);
    }

    // Liczymy CRC
    uint32_t crcVal = computeCRC32(memBuf.data(), memBuf.size());

    // Zapisujemy dane
    outputFile.write((const char*)memBuf.data(), memBuf.size());

    // Dopisujemy 4 bajty CRC
    unsigned char crcBytes[4];
    crcBytes[0] = (unsigned char)(crcVal & 0xFF);

```

```

        crcBytes[1] = (unsigned char)((crcVal >> 8) & 0xFF);
        crcBytes[2] = (unsigned char)((crcVal >> 16) & 0xFF);
        crcBytes[3] = (unsigned char)((crcVal >> 24) & 0xFF);
        outputFile.write((const char*)crcBytes, 4);

        outputFile.close();
    }

    // Funkcja wyswietlajaca zakodowane dane w formacie (offset, length, nextChar)
    void printCompressedData(const vector<Match>& compressed) {
        for (const auto& match : compressed) {
            cout << "(" << match.offset << ", " << match.length << ", "
                << (match.nextChar ? match.nextChar : ' ') << ")" << "\n";
        }
    }

    // Funkcja obliczajaca stopien kompresji
    double calculateCompressionRatio(const string& inputFileName,
        const string& outputFileName) {
        streampos inputSize, outputSize;

        ifstream inputFile(inputFileName, ios::binary | ios::ate);
        if (inputFile.is_open()) {
            inputSize = inputFile.tellg();
            inputFile.close();
        }
        else {
            cerr << "Nie mozna otworzyc pliku wejscowego." << endl;
            return -1;
        }

        ifstream outputFile(outputFileName, ios::binary | ios::ate);
        if (outputFile.is_open()) {
            outputSize = outputFile.tellg();
            outputFile.close();
        }
        else {
            cerr << "Nie mozna otworzyc pliku wyjscowego." << endl;
            return -1;
        }

        if (inputSize == 0) {
            return -1;
        }
        return (double)outputSize / (double)inputSize;
    }

    int main() {
        string inputFileName;
        string outputFileName;
        int dictSize;
        int bufferSize;
    }

```

```

//Pobranie danych od uzytkownika
cout << "Podaj nazwe pliku wejscowego: ";
cin >> inputFileName;

cout << "Podaj nazwe pliku wyjscowego: ";
cin >> outputFileName;

do {
    cout << "Podaj rozmiar slownika: ";
    cin >> dictSize;
    if (dictSize <= 0) {
        cout << "Slownik nie moze byc zerowy lub ujemny.\n";
    }
} while (dictSize <= 0);

do {
    cout << "Podaj rozmiar bufora: ";
    cin >> bufferSize;
    if (bufferSize <= 0) {
        cout << "Bufor nie moze byc zerowy lub ujemny.\n";
    }
} while (bufferSize <= 0);

ifstream inputFile(inputFileName, ios::binary);
if (!inputFile.is_open()) {
    cerr << "Nie mozna otworzyc pliku " << inputFileName << endl;
    return 1;
}

string inputText((istreambuf_iterator<char>(inputFile)), istreambuf_iterator<char>());
inputFile.close();

// Kompresja
vector<Match> compressed = compressLZ77(inputText, dictSize, bufferSize);

// Zapis i wyswietlenie wyniku
saveAndPrintCompressed(compressed, outputFileName, dictSize, bufferSize);

double compressionRatio = calculateCompressionRatio(inputFileName, outputFileName);
if (compressionRatio >= 0) {
    cout << "Stopien kompresji: " << compressionRatio << endl;
}

// printCompressedData(compressed);

return 0;
}

```


Dekoder:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <cmath>
#include <cstdint>

using namespace std;

// Struktura dopasowania
struct Match {
    int offset; // Przesuniecie w slowniku
    int length; // Dlugosc dopasowania
    char nextChar; // Nastepny znak
};

//Obliczenie CRC32
uint32_t computeCRC32(const unsigned char* data, size_t length) {
    uint32_t crc = 0xFFFFFFFF;
    uint32_t polynomial = 0xEDB88320; // typowy wielomian

    for (size_t i = 0; i < length; ++i) {
        crc ^= data[i];
        for (int b = 0; b < 8; b++) {
            if (crc & 1) {
                crc = (crc >> 1) ^ polynomial;
            }
            else {
                crc >>= 1;
            }
        }
    }
    return crc ^ 0xFFFFFFFF;
}

int decodeVariableLength(const unsigned char* buf, size_t& pos, size_t bufSize, int maxValue)
{
    // Obliczamy liczbe bitow i bajtow potrzebnych do reprezentacji wartosci
    int numBits = (maxValue > 0) ? (int)ceil(log2(maxValue + 1)) : 1;
    int byteCount = (numBits + 7) / 8;

    if (pos + byteCount > bufSize) {
        cerr << "Bład: niekompletne dane dla zmiennej liczby bitow" << endl;
        return 0; // Wartosc domyslna w przypadku bledu
    }

    int value = 0;

    // Skladamy wartosc z wczytanych bajtow (zakladamy little-endian)
    for (int i = 0; i < byteCount; i++) {
```

```

        value |= (buf[pos + i] << (8 * i));
    }

    // Aktualizujemy pozycje
    pos += byteCount;
    return value;
}

// Wczytanie skompresowanych danych bez CRC
vector<Match> readCompressedData(const vector<unsigned char>& rawData,
    int dictSize, int bufferSize) {
    vector<Match> compressed;
    size_t pos = 0;
    size_t bufSize = rawData.size(); // Rozmiar bufora

    while (pos < bufSize) {

        Match temp;

        temp.offset = decodeVariableLength(rawData.data(), pos, bufSize, dictSize);
        if (pos >= bufSize) break;

        temp.length = decodeVariableLength(rawData.data(), pos, bufSize, bufferSize);
        if (pos >= bufSize) break;

        if (pos >= bufSize) break;
        temp.nextChar = (char)rawData[pos];
        pos++;

        compressed.push_back(temp);
    }
    return compressed;
}

// Dekompresja LZ77
string decompressLZ77(const vector<Match>& compressed) {
    string decompressed;
    for (const auto& match : compressed) {
        if (match.length > 0) {
            //Sprawdzamy czy offset jest poprawny
            if (match.offset <= 0 || (size_t)match.offset > decompressed.size()) {
                cerr << "Blad dekodowania: offset poza zakresem. offset=" << match.offset
                    << ", zdekodowany rozmiar=" << decompressed.size() << "\n";
                break; // Niepoprawne dane
            }
            int start = (int)decompressed.size() - match.offset;
            //Sprawdzamy czy mamy wystarczajaco znakow od skopiowania
            for (int i = 0; i < match.length; ++i) {
                int index = start + i;
                if (index < 0 || (size_t)index >= decompressed.size()) {

```

```

        cerr << "Bład dekodowania: start poza zakresem (start=" << start << ")\\n";
        break;
    }
    decompressed.push_back(decompressed[index]);
}
}
if (match.nextChar != 0) {
    decompressed.push_back(match.nextChar);
}
}
return decompressed;
}

```

```

int main() {
    string inputFileName;
    string outputFileName;
    int dictSize;
    int bufferSize;

    // Pobranie danych od uzytkownika
    cout << "Podaj nazwe pliku wejscowego: ";
    cin >> inputFileName;

    cout << "Podaj nazwe pliku wyjscowego: ";
    cin >> outputFileName;

    do {
        cout << "Podaj rozmiar slownika (ten sam co podczas kompresji): ";
        cin >> dictSize;
        if (dictSize <= 0) {
            cout << "Slownik nie moze byc zerowy lub ujemny.\\n";
        }
    } while (dictSize <= 0);

    do {
        cout << "Podaj rozmiar bufora (ten sam co podczas kompresji): ";
        cin >> bufferSize;
        if (bufferSize <= 0) {
            cout << "Bufor nie moze byc zerowy lub ujemny.\\n";
        }
    } while (bufferSize <= 0);

    ifstream inFile(inputFileName, ios::binary | ios::ate);
    if (!inFile.is_open()) {
        cerr << "Nie mozna otworzyc pliku " << inputFileName << endl;
        return 1;
    }

    streampos fileSize = inFile.tellg();
    if (fileSize < 4) {
        cerr << "Plik za maly, brak miejsca na CRC." << endl;
        return 1;
    }
}

```

```

}

// Wczytujemy wszystkie dane
vector<unsigned char> fileData(fileSize);
inFile.seekg(0, ios::beg);
inFile.read(reinterpret_cast<char*>(fileData.data()), fileSize);
inFile.close();

// Odczytujemy ostatnie 4 bajty jako CRC
uint32_t readCRC = 0;
size_t offsetCRC = fileData.size() - 4;
readCRC = fileData[offsetCRC];
readCRC |= (fileData[offsetCRC + 1] << 8);
readCRC |= (fileData[offsetCRC + 2] << 16);
readCRC |= (fileData[offsetCRC + 3] << 24);

// Dane skompresowane to wszystko oprócz ostatnich 4 bajtów
vector<unsigned char> compData(fileData.begin(),
    fileData.begin() + offsetCRC);

// Liczymy CRC
uint32_t calcCRC = computeCRC32(compData.data(), compData.size());

// Porównujemy
if (calcCRC != readCRC) {
    cerr << "BLAD: plik uszkodzony! CRC sie nie zgadza." << endl;
    return 1;
}

// Wczytanie skompresowanych danych
vector<Match> compressedData = readCompressedData(compData, dictSize, bufferSize);
if (compressedData.empty()) {
    cerr << "Brak danych lub bledny plik." << endl;
    return 1;
}

// Dekompresja danych
string decompressedData = decompressLZ77(compressedData);

// Zapis zdekodowanych danych do pliku wyjsciowego
ofstream outFile(outputFileName, ios::binary);
if (!outFile.is_open()) {
    cerr << "Nie mozna otworzyc pliku wyjsciowego: "
        << outputFileName << endl;
    return 1;
}
outFile.write(decompressedData.data(), (streamsize)decompressedData.size());
outFile.close();

cout << "Dekompresja zakonczona poprawnie.\n";
cout << "Odtworzony tekst:\n" << decompressedData << endl;
return 0;}

```